

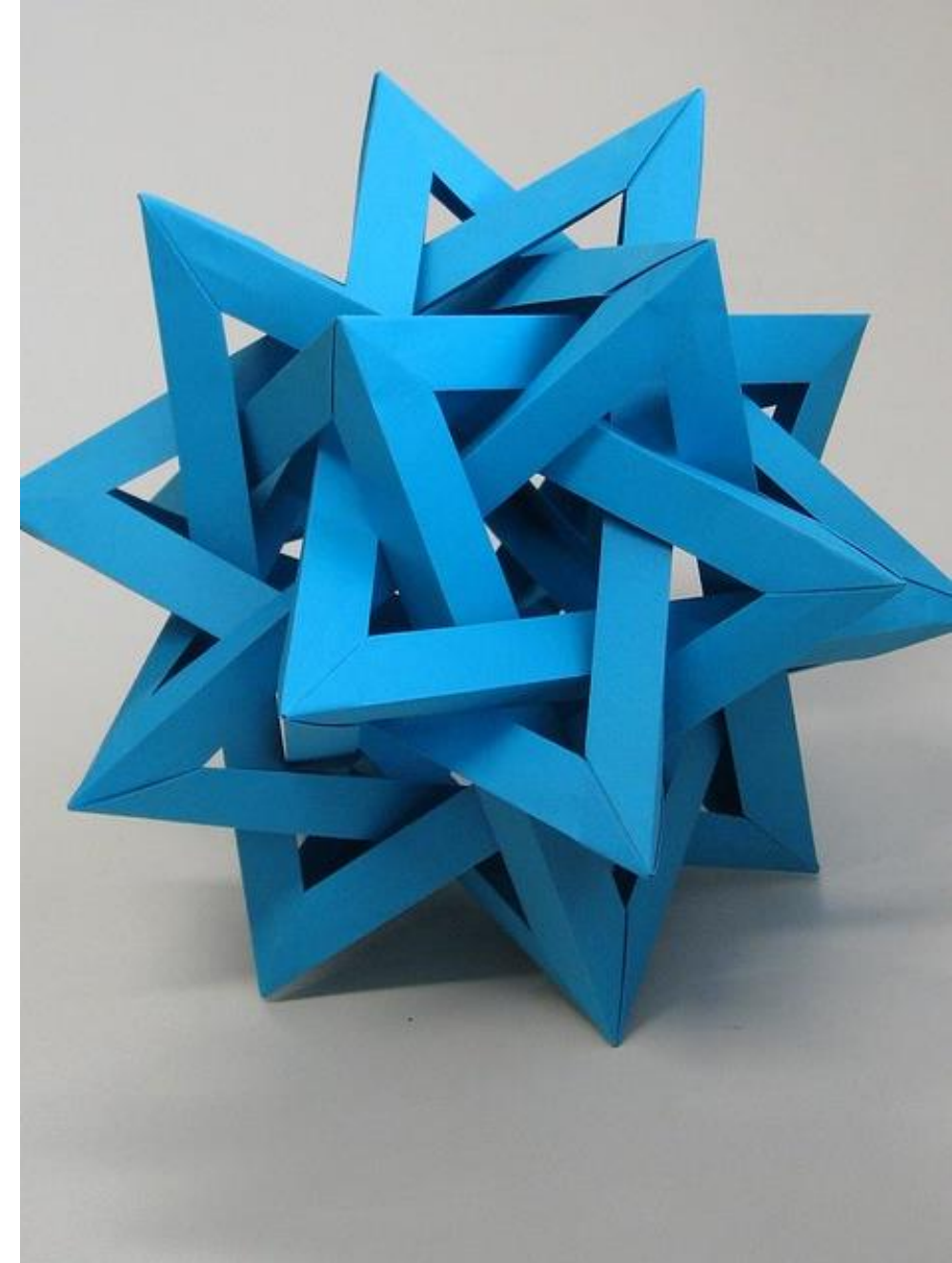


Unit P7: Files and Error management

READING AND WRITING FILES, EXTRACTING
FILE CONTENT, MANAGING ERRORS AND
EXCEPTIONS



Chapter 7



Unit Goals

- To read and write text files
- To process collections of data coming from files
- To raise and handle exceptions (errors)

In this unit, you will learn how to write programs that manipulate files

Reading and Writing Text Files



7.1

Reading and Writing Text Files

- Text files are very commonly used to store information
 - They are the most 'portable' types of data files
- Examples of text files include
 - Files that are created with a simple text editor, such as Windows Notepad,
 - Python source code
 - HTML files
 - CSV files (comma-separated)
 - ...

Extra: File Types

- Text Files = **files whose content is a sequence of characters**
 - ASCII or UNICODE
- Non-text files are called **binary files**
 - They contain a sequence of bits whose interpretation is not simply as ASCII/UNICODE characters but follows a more complicated encoding, usually defined by the program that generated them (and will read them).
- Examples of binary files:
 - .doc/docx files:
 - “understandable” only by the program MS Word (or equivalent)
 - The binary content follows an encoding defined by Word, to represent documents with formatted text, images, etc.
 - .pdf files:
 - Understandable by a PDF reader program (who understands the relative encoding)
- What happens if we treat them as text files?

Handling Files

- Three main steps:
 - **Opening the file**
 - **Accessing the file (reading/writing)**
 - **Closing the file.**

Opening Files: Read Mode

- To access a file, you must first open it
- Suppose you want to read data from a file named `input.txt`, located in the same directory as your program
- To open a file for reading, we use the **`open()`** function, providing the name of the file (a string) as the first argument, and the string `"r"` as the second argument:

```
infile = open("input.txt", "r")
```

- A “**file object**” is returned, that will be used for accessing the file content

Opening Files: Read Mode (2)

- Important things to keep in mind:
 - When opening a file for reading, **the file must exist** (and be accessible) or an error (exception) occurs
 - The file object returned by the open function must be **saved in a variable**
 - All operations for accessing a file are made via the file object

Opening Files: Read Mode (3)

```
infile = open("input.txt", "r")
```



Variable of type "file"

- This operation creates a connection between the actual file ("input.txt", located in your secondary memory), and a data structure in main memory (logic file, `infile`) that allows you to use the data contained in the file in your program.

Opening Files: Write Mode

- To open a file for writing, you provide the name of the file as the first argument to the open function and the string "w" as the second argument:

```
outfile = open("output.txt", "w")
```

- If the output file already exists, **it is emptied** before the new data is written into it
- If the file does not exist, an empty file is created
 - You need permission to write in that directory.

Opening files: Append Mode

- Alternatively, you can add content “at the end” of an existing file, by opening it in append mode:

```
outfile = open("output.txt", "a")
```

- Careful:
 - If the file does not exist, it is created empty
 - If the file already exists, content is added at the endf.

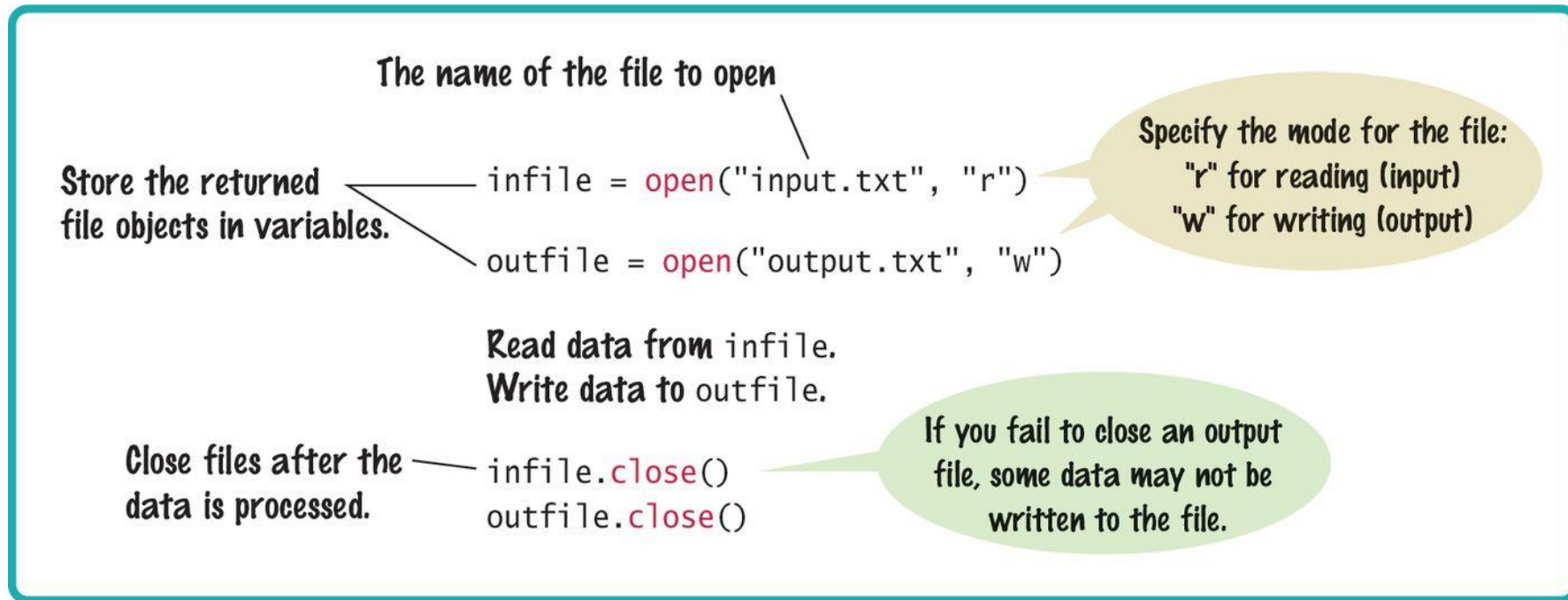
Closing Files: Important

- When you are done processing a file, be sure to close the file using the `close()` method:

```
infile.close()  
outfile.close()
```

- If your program exits without closing a file that was opened for writing, **some of the output may not be written to the physical file**

Syntax: Opening And Closing Files



File Opening Modes

Mode	Description
r	Opens a file for reading. (default)
w	Opens a file for writing. Creates a new file if it does not exist or truncates the file if it exists.
x	Opens a file for exclusive creation. If the file already exists, the operation fails.
a	Opens a file for appending at the end of the file without truncating it. Creates a new file if it does not exist.
t	Opens in text mode. (default)
b	Opens in binary mode.
+	Opens a file for updating (reading and writing)

Where is the file?

- You program looks for files in the directory in which **Python is run**.
 - With VSCode, this is the **main folder of your open project**.
 - If the file is placed in a different folder, it will not be found
 - `FileNotFoundError: [Errno 2] No such file or directory: 'input.txt'`

```
infile = open("input.txt", "r")
```

⚠ Reading UTF-8 files

- If the file contains **accented letters or other symbols not present in the basic ASCII code**, it is usually encoded with Unicode encoding, specifically UTF-8.
- The `open()` function in Python uses the *encoding defined by the operating system by default*.
 - It depends on the operating system and the installation language!
- To specify that the file is UTF-8 (overruling the default operating system encoding), **always add** an argument: `encoding='utf-8'` to `open()`

```
infile = open('file.txt', 'r', encoding='utf-8' )
```

Reading from a file

IMPORTANT!!!

- Before analyzing the methods used to *read* data from a file, it is essential to discuss the general **access type**
- Python (like other languages) allows several types of access but we will consider only one (the easiest) → **sequential access:**
 1. When opening the file, an imaginary "*cursor*" is placed at the beginning of the file.
 2. The file can then be read:
 - Only from the beginning and going "forward" (you cannot go back)
 - One "datum" at a time (you cannot jump around)

Reading from a file

IMPORTANT!!!

- Once the access type (sequential) has been defined, we can collect data in "blocks" of 4 different sizes:
 - One line*: **`infile.readline()`** / **`for line in infile`**
 - 1 character: `infile.read(1)`
 - N characters: `infile.read(N)`
 - The entire file: `infile.readlines()` / `infile.read()`
- After each read, the cursor is automatically advanced to the first unread "block".

* Line = string terminated by carriage return '\n'

Reading a **Line** From a File

- To read **a line of text** from a file, call the `readline()` method on the file object that was returned by `open()`:

string



```
line = infile.readline()
```

- The `readline()` method reads the text, starting at the current position of the “cursor” and continuing **until the end of the line is encountered**
 - **The end of the line is identified by the character ‘\n’, which is included in the read string.**
 - The cursor is then moved to the beginning of the next line

Reading a **Line** From a File (2)

- For example, suppose `input.txt` contains the lines
flying
circus
- The first call to `readline()` returns the string `"flying\n"`
 - Remember that `\n` denotes the newline character that marks the end of the line
- If you call `readline()` a second time, it returns the string `"circus\n"`
- **VERY IMPORTANT** to remember that the cursor advances automatically (every time you call the readline method you get a different line)

Reading a **Line** From a File (3)

- Calling `readline()` again yields the empty string `""` because you have reached the **end of the file**
- If the file contains a blank line, then `readline()` returns a string containing only the newline character `"\n"`

Remember:

End of file → `input.readline() == ''`

Empty line → `input.readline() == '\n'`

Reading Multiple Lines From a File

- You **repeatedly** read a line of text and process it **until** you reach the end of the file
- You can use an empty string as **sentinel value**, since this is the value returned by the `readline()` method after the **end of file** has been reached

```
line = infile.readline()
while line != "":
    # Process the line.
    line = infile.readline()
```

NOTE: Remember that `line` always includes the trailing `'\n'`

Converting data Read from a File

- Like the input function, the `readline()` method always returns a **string**
- If the file contains **numerical** data, the strings must be **converted** to the numerical value using the `int()` or `float()` function:

```
value = float(line)
```

- The newline character at the end of the line is ignored when the string is converted to a numerical value
 - Example: `x = int("123\n")` => **x contains 123** (integer)

«Automatic» Line-by-Line Reading

- A file opened for reading can also be seen as **a container of strings**, where each **line** (including the `'\n'`) corresponds to one element of the container.
- So, we can also use it **for...in loop**. General pattern:

```
for line in infile :  
    # work on line
```

- At the beginning of each iteration of the loop, the **line variable** is assigned the value of a string containing the next line in the file.
 - Key difference between a file and a “normal” container: once you read the file you must **close** it before you can iterate over it again

«Automatic» Line-by-Line Reading

- We therefore have two ways to read a file "by lines"
 - More compact and preferred in most cases...

```
for line in infile :  
    # process the line
```

In both cases
`line` keeps the '\n'

- Necessary in some special cases (see later):

```
line = infile.readline()  
while line != "" :  
    # process the line  
    line = infile.readline()
```

Writing To A File

- We can write a string into a file (open in write-mode) using the `write()` method:

```
outfile.write("Hello, World!\n")
```

- Unlike `print()`, when writing text to an output file, you must **explicitly write the newline** character if we want to include it.
- You can also write **formatted strings** to a file with the `write()` method:

```
outfile.write(f'Number of entries: {count}\nTotal: {total:%8.2f}\n')
```

Example: File Reading/Writing

- Suppose you are given a text file that contains a sequence of floating-point values, stored one value per line
- You need to read the values and write them to a new output file, with two fractional digits, and followed by their total and average value
- If the **input** file has the contents

32.0

54.0

67.5

80.25

115.0

Example: File Reading/Writing (2)

- The **output** file should contain

```
32.00
54.00
67.50
80.25
115.00
-----
Total: 348.75
Average: 69.75
```

Example 2: Reading and writing files

- Suppose we have a text file containing a sequence of integers, stored one per line.
- You want to read these values and write them to a new output file, filtering out all values less than a threshold **D**
- For example, if the input file is

54

67

32

80

115

And the threshold is 60 the created file will contain only the 2nd, 4th and 5th values.

Common Error

■ Backslashes in File Names

- When using a String literal for a file name with path information, you need to supply each backslash twice:

```
infile = open("c:\\homework\\input.txt", "r")
```

- A single backslash inside a quoted string literal is the *escape character*, which means the next character is interpreted differently (for example, '\n' for a newline character)
- When a user supplies a filename into a program, the user does not need to type the backslash twice

In this course, you can avoid these issues by keeping all your files in the same folder as your Python program.

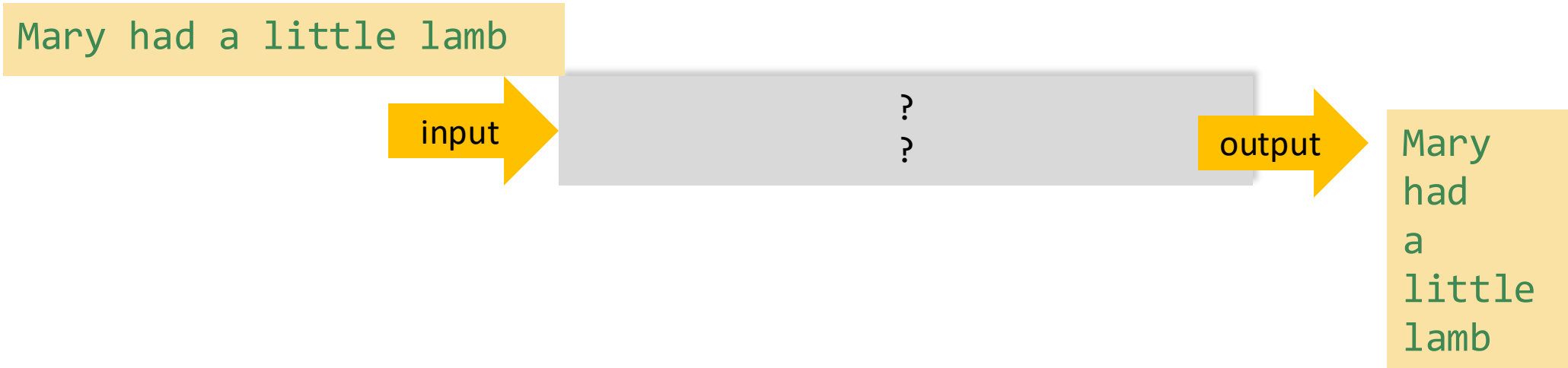
Free Text File I/O



7.2

Text File Input and Output

- How to process files with complex contents?
- For some problems, the “unit of data” might not be a line, but something else (e.g., a single **word**).
- For example, what if I want to do this?



Processing Text File Input

- There are times when you want to read input by:
 - Each word
 - Each line
 - A single character
- Python provides methods such as: `read()`, `split()` and `strip()` for these tasks

Processing text input is required for almost all types of programs that interact with the user

Cleaning up strings read from files: strip() and variants

- Recall that each input line ends with a newline (`\n`) character. E.g.:

s p a m \n

- Generally, the newline character must be removed before the input string is used
- To remove the newline character, apply the `rstrip()` string method:

```
line = line.rstrip()
```

- This results in the string:

s p a m

Character Strip Methods

Table 1 Character Stripping Methods

Method	Returns
<code>s.lstrip()</code> <code>s.lstrip(chars)</code>	A new version of <i>s</i> in which white space (blanks, tabs, and newlines) is removed from the left (the front) of <i>s</i> . If provided, characters in the string <i>chars</i> are removed instead of white space.
<code>s.rstrip()</code> <code>s.rstrip(chars)</code>	Same as <code>rstrip</code> except characters are removed from the right (the end) of <i>s</i> .
<code>s.strip()</code> <code>s.strip(chars)</code>	Similar to <code>rstrip</code> and <code>rstrip</code> , except characters are removed from the front and end of <i>s</i> .

Character Strip Examples

Table 2 Character Stripping Examples

Statement	Result	Comment
<pre>string = "James\n" result = string.rstrip()</pre>	J a m e s	The newline character is stripped from the end of the string.
<pre>string = "James \n" result = string.rstrip()</pre>	J a m e s	Blank spaces are also stripped from the end of the string.
<pre>string = "James \n" result = string.rstrip("\n")</pre>	J a m e s	Only the newline character is stripped.
<pre>name = " Mary " result = name.strip()</pre>	M a r y	The blank spaces are stripped from the front and end of the string.
<pre>name = " Mary " result = name.lstrip()</pre>	M a r y	The blank spaces are only stripped from the front of the string.

Reading Words

- Sometimes you may need to read the individual words from a text file
- For example, suppose our input file contains two lines of text
Mary had a little lamb,
whose fleece was white as snow

Reading Words (2)

- We would like to print to the terminal, one word per line

Mary

had

a

little

. . .

- Because there is no method for reading a word from a file, you must first read a **line** and then **split** it into individual words
 - We have already seen **split()** in its basic version

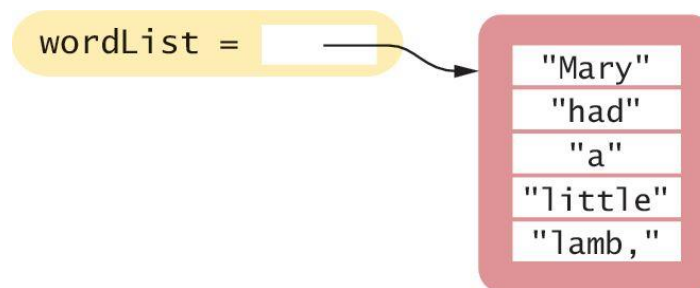
```
line = line.rstrip()  
wordlist = line.split()
```

Reading Words (3)

- The `split()` method returns the list of substrings that results from splitting the string at each blank space (or tabs or newlines)
- For example, if line contains the string:

line = M a r y h a d a l i t t l e l a m b ,

- It will be split into 5 substrings that are stored in a list in the same order in which they occur in the string:



Reading Words (4)

- Notice that the **last** word in the line contains a **comma**
- If we only want to print the words contained in the file **without punctuation marks**, we can **strip those** from the substrings using the **rstrip()** method:

```
word = word.rstrip(".,?!")
```

Reading Words: Complete Example

```
inputFile = open("lyrics.txt", "r")
for line in inputFile :
    line = line.rstrip()
    wordList = line.split()
    for word in wordList :
        word = word.rstrip(". , ? !")
        print(word)

inputFile.close()
```

split()

```
str.split(sep=None, maxsplit=-1)
```

Return a list of the words in the string, using *sep* as the delimiter string. If *maxsplit* is given, at most *maxsplit* splits are done (thus, the list will have at most `maxsplit+1` elements). If *maxsplit* is not specified or `-1`, then there is no limit on the number of splits (all possible splits are made).

If *sep* is given, consecutive delimiters are not grouped together and are deemed to delimit empty strings (for example, `'1,2'.split(',')` returns `['1', '', '2']`). The *sep* argument may consist of multiple characters (for example, `'1<>2<>3'.split('<>')` returns `['1', '2', '3']`). Splitting an empty string with a specified separator returns `['']`.

For example:

```
>>> '1,2,3'.split(',')
['1', '2', '3']
>>> '1,2,3'.split(',', maxsplit=1)
['1', '2,3']
>>> '1,2,,3'.split(',')
['1', '2', '', '3', '']
```

If *sep* is not specified or is `None`, a different splitting algorithm is applied: runs of consecutive whitespace are regarded as a single separator, and the result will contain no empty strings at the start or end if the string has leading or trailing whitespace. Consequently, splitting an empty string or a string consisting of just whitespace with a `None` separator returns `[]`.

For example:

```
>>> '1 2 3'.split()
['1', '2', '3']
>>> '1 2 3'.split(maxsplit=1)
['1', '2 3']
>>> ' 1 2 3 '.split()
['1', '2', '3']
```

<https://docs.python.org/3/library/stdtypes.html#str.split>

Additional String Splitting Methods

Table 3 String Splitting Methods

Method	Returns
<code>s.split()</code> <code>s.split(<i>sep</i>)</code> <code>s.split(<i>sep</i>, <i>maxsplit</i>)</code>	Returns a list of words from string <i>s</i> . If the string <i>sep</i> is provided, it is used as the delimiter; otherwise, any white space character is used. If <i>maxsplit</i> is provided, then only that number of splits will be made, resulting in at most <i>maxsplit</i> + 1 words.
<code>s.rsplit(<i>sep</i>, <i>maxsplit</i>)</code>	Same as <code>split</code> except the splits are made starting from the end of the string instead of from the front.
<code>s.splitlines()</code>	Returns a list containing the individual lines of a string split using the newline character <code>\n</code> as the delimiter.

Additional String Splitting Examples

Table 4 String Splitting Examples

Statement	Result	Comment
<pre>string = "a,bc,d" string.split(",")</pre>	"a" "bc" "d"	The string is split at each comma.
<pre>string = "a b c" string.split()</pre>	"a" "b" "c"	The string is split using the blank space as the delimiter. Consecutive blank spaces are treated as one space.
<pre>string = "a b c" string.split(" ")</pre>	"a" "b" "" "c"	The string is split using the blank space as the delimiter. With an explicit argument, the consecutive blank spaces are treated as separate delimiters.
<pre>string = "a:bc:d" string.split(":", 2)</pre>	"a" "bc:d"	The string is split into 2 parts starting from the front. The split is made at the first colon.
<pre>string = "a:bc:d" string.rsplit(":", 2)</pre>	"a:bc" "d"	The string is split into 2 parts starting from the end. The split is made at the last colon.

Exercise

- Read a file “`estrema.dat`” containing pairs of integers x, y one pair per line and separated by commas, and create a second file “`differences.dat`” containing the value of the differences $x-y$, one per line.
- Example:

```
23,32
```

```
2,11
```

```
19,6
```

```
23,5
```

```
3,2
```

```
...
```

`extremes.dat`

```
-9
```

```
-9
```

```
13
```

```
18
```

```
1
```

```
...
```

`differences.dat`

Reading from a file (Reprise)

- Once the access type (sequential) has been defined, we can collect data in "blocks" of 4 different sizes:
 - One line*: `infile.readline()` / `for line in infile`
 - 1 character: **`infile.read(1)`**
 - N characters: **`infile.read(N)`**
 - The entire file: `infile.readlines()` / `infile.read()`

Reading Single Characters

- The `read()` method may take a single **argument** that specifies the **number of characters** to read
- The method returns a string containing the characters
- When supplied with an argument of **1**, the `read()` method returns a string consisting of **the next character** in the file

```
char = inputFile.read(1)
```

- If the end of the file is reached, it returns an empty string `""`

Algorithm: Reading Characters

```
char = inputFile.read(1)
while char != "" :
    Process character
    char = inputFile.read(1)
```

Reading from a file (Reprise)

- Once the access type (sequential) has been defined, we can collect data in "blocks" of 4 different sizes:
 - One line*: `infile.readline()` / `for line in infile`
 - 1 character: `infile.read(1)`
 - N characters: `infile.read(N)`
 - The entire file: **`infile.readlines()` / `infile.read()`**

Useful but not strictly necessary. “Shortcuts” for stuff that can be done using just the previous methods.

Reading the whole file – as a “Huge” String

- The `read()` method without any parameter will read the entire file as a single string

```
contents = infile.read()  
# the whole file is read in a single (huge!) string
```

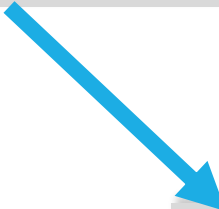
- We we want to split the huge string into individual lines we can use:

```
lines = contents.split('\n')
```

Reading the whole file – as a List of Strings

- The `readlines()` method will read the entire file as a **list of strings** (one string per line)

```
lines = infile.readlines()
```



```
# equivalent to  
lines = []  
for line in infile:  
    lines.append(line)
```

Processing Text Files Example

PAGE 319 – 440

Steps to Processing Text Files

■ Problem Statement:

- Read two country data files, `worldpop.txt` and `worldarea.txt`
- Write a file `world_pop_density.txt` that contains country names and population densities with the country names aligned left and the numbers aligned right

```
Afghanistan 647500
Akrotiri    123
Albania     28748
Algeria     2381740
American Samoa 199
. . .
```

`worldarea.txt` (area of each country)

```
Afghanistan 32738376
Akrotiri    15700
Albania     3619778
Algeria     33769669
American Samoa 57496
. . .
```

`worldpop.txt` (population of each country)

Six Steps to Processing Text Files

- Understand the Processing Task
 - **Process the data “on the go” or store data and then process?**
- Determine which files you need to read and write
- Choose a mechanism for obtaining the file names
- Choose between iterating over the file (`for..in file`) or reading individual lines (`while file.readline()`)
 - If all data is on one line, normally use line input
- With line-oriented input, extract required data
 - Examine the line and handle whitespace, delimiters...
- Use functions to factor out common tasks

Step 1: Understand the Task

- While there are more lines to be read
 - Read a line from each file
 - Extract the country name
 - population = number following the country name in the first file
 - area = number following the country name in the second file
 - If area != 0
 - density = population / area
 - Write country name and density on output file

Afghanistan	50.56
Akrotiri	127.64
Albania	125.91
Algeria	14.18
American Samoa	288.92
• • •	

Step 2: Determine the files

- Determine the file you need to read and write
- There are two input files:
 - worldpop.txt
 - worldarea.txt
- There is one output file:
 - world_pop_density.txt

Step 3: Obtaining file names

- Choose a mechanism for obtaining the file names
- We have three options:
 - Hardcode the filename as a literal string
 - Prompt the user
 - Use a command line argument (**advanced topic, will not be treated in this course**)
- Since the request does not specify which method to use, we will use hard-coded files names
 - The file names are constants, in this example

Step 4: Iterate or read lines?

- Choose between **for...in** or **while line != ""**
- Generally, if the data is grouped in lines, we iterate over the file (**for...in**)
- If the data is spread over several lines, we read the individual lines (**while line != ""**)
- In this example we will read **individual lines** since we are reading alternated lines from two input files

Step 5: Extract data from lines

- Extract the data into the individual fields
 - Use split, rsplit, string slices, etc., to extract the data elements

Step 6: Break up in functions

- Use functions to factor out common tasks
- Find the repetitive tasks and develop functions to handle them

EXTRA

- What if the names of the countries are not in the same order in the two files??

Structured Text Files



7.2

FILES CONTAINING “DATA RECORDS”

Reading composite data (records)

- As already seen in some examples, a file often contains a **sequence of data composed of**
 - **A series of lines all with the same format**
 - **Each row** (*record*) contains several **fields** (*field*)
 - **The fields are separated by a specific *separator***
- Example:
 - A file contains student data, whose records are composed of
 - ID, name, address and year of registration, separated by a comma
 - This type of file can be exported, for example, from MS Excel

Reading composite data (records)

- There may be particular cases that complicate management
 - Example: using the "space" separator, the following file would not have a fixed format

Germany 200

France 220

Italy 300

United Kingdom 175

...

▮ These are 3 fields if I consider the space as a separator!!!

- For this reason, we tend to use an unambiguous separator.

Germany,200

France,220

Italy,300

United Kingdom,175

Reading composite data (records)

★ General pattern of reading a (structured) record file

For each line of the file:
read the record
break it down into its fields
process the fields

One of the two line
reading patterns

`split()`
`+ r/strip()`

CSV format

★CSV

- Comma Separated Values

★Common as data table interchange formats

★They can also be read/written with Excel and similar programs

	A	B	C	D
1	Detective Story	1951	William Wyler	
2	Airport 1975	1974	Jack Smight	
3	Hamlet	1996	Kenneth Branagh	
4	American Beauty	1999	Sam Mendes	
5	Bitter Moon	1992	Roman Polanski	
6	Million Dollar Baby	2004	Clint Eastwood	
7	Round Midnight	1986	Bertrand Tavernier	
8	Kiss of the Spider Woman	1985	Héctor Babenco	
9	Twin Falls Idaho	1999	Michael Polish	
10	Traffic	2000	Steven Soderbergh	
11				

CSV

Detective Story, 1951, William Wyler
Airport 1975,1974,Jack Smight
Hamlet,1996,Kenneth Branagh
American Beauty, 1999, Sam Mendes
Bitter Moon, 1992, Roman Polanski
• • •

Exercise

- ✦ **data.csv** file containing a series of records each consisting of comma-separated fields, store them in a table.
- ✦ The goal is to understand how to 'memorize' a file entirely and make it accessible in memory
- ✦ It won't be much different than filling the table with a series of values (like a list of lists)

FirstName, LastName, Age, Score

John, Doe, 23, C+

Woman, Wilson, 24, B-

Sarah, Parker, 22, A-

Jack, McCormick, 23, B+



First Name	Last Name	Age	Score
John	Doe	23	C+
Woman	Wilson	24	B-
Sarah	Parker	22	TO-
Jack	McCormick	23	B+

Exception Handling 7.4

Exception Handling

- There are two aspects to dealing with run-time program errors:
 - Detecting Errors
 - Handling Errors
- **Example:** the `open()` function can detect an attempt to read from a non-existing file
 - The open function cannot handle the error (it doesn't know what to do!!)
- **Exception handling** provides a flexible mechanism for passing control from the error to a handler that can deal with it

Exception Handling: Overview

DETECTING ERRORS (RAISE)

- The program should **check** whether an error condition occurs
- In that case, the program **raises an exception**
- Statement: **raise**
- There are different **types** of exceptions (ValueError, IOError, ...) depending on the cause
- The exception may also contain a **message** describing the problem
- **This is what the open() function (and other functions we've used) do.**

HANDLING ERRORS (TRY...EXCEPT)

- If you know that some code may generate exceptions, you must define some **handler code**
- The “controlled” code (i.e. the code that might generate an error) is put inside a **try** block
- The “handler” code is put inside an **except** block
- Failure to handle an exception will **stop** the program

Exception Handling: Overview

DETECTING ERRORS (RAISE)

- The program should **check** whether an error condition occurs
- In that case, the program **raises an**

Principle:

Do → Check → Report

(ValueError, IOError, ...) depending on the cause

- The exception may also contain a **message** describing the problem
- **This is what the open() function (and other functions we've used) do.**

HANDLING ERRORS (TRY...EXCEPT)

- If you know that some code may generate exceptions, you must define some **handler code**

■ The “controlled” code (i.e. the code that

Principle:

Try → Intercept → Manage

- The handler code is put inside an **except** block
- Failure to handle an exception will **stop** the program

Handling Exceptions

- Every exception should be handled somewhere in the program
- This is a **very complex problem**
 - You need to handle each possible exception and react to it appropriately
 - Not all errors are recoverable
- Handling recoverable errors:
 - Simplest approach: terminate the program
 - More user-friendly: Ask the user to correct the error
- **We'll see a couple of contexts where it's important to handle exceptions**

Handling Exceptions: Try-Except

- In the simple programs that we've seen in this course, the following are **two important types of errors** to intercept.
- Opening file failure with `open()`
→ may throw **IOError**
- Type conversion failure with `int()` `float()`
→ may throw **ValueError**

Other Examples of Exceptions (Try them)

<code>val/0</code>	<code>ZeroDivisionError</code> : division by zero
<code>int('pippo')</code>	<code>ValueError</code> : invalid literal for int() with base 10: 'pippo'
<code>a</code>	<code>NameError</code> : name 'a' is not defined
<code>l[10]</code>	<code>IndexError</code> : list index out of range
<code>d['pippo']</code>	<code>KeyError</code> : 'pippo'
<code>open('pippo.txt')</code>	<code>FileNotFoundError</code> : [Errno 2] No such file or directory: 'pippo.txt'

Handling Exceptions: Try-Except

- You handle exceptions with the `try/except` statement
- The `try` block contains one or more statements that *may cause* an exception of the kind that you are willing to handle
- Each `except` clause contains the *handler* for an exception type
 - **You need to know the type of error that is generated!**

```
try :  
    instructions that can generate exceptions  
...  
except exception type :  
    instructions for handling the exception
```

Try-Except: An Example

```
try :
    filename = input("Enter filename: ")
    infile = open(filename, "r")
    line = infile.readline()
    value = int(line)
    . . .
except IOError :
    print("Error: file not found.")
except ValueError as exception :
    print("Error:", str(exception))
```

`open()` can raise an `IOError` exception

`int()` can raise a `ValueError` exception

Execution transfers here if file cannot be opened

Execution transfers here if the string cannot be converted to an int

***If either of these exceptions is raised,
the rest of the instructions in the try
block are skipped***

Example

- If an `IOError` exception is raised, the `except` clause for the `IOError` exception is executed
- If a `ValueError` exception occurs, then second `except` clause is executed
- If any other exception is raised it **will not be handled** by any of the `except` blocks

EXTRA: Output Messages

- When the body of this handler is executed, it prints the message included with the exception

```
except ValueError as exception :  
    print("Error:", str(exception))
```

- For example, if the string passed to the `int()` function was `"35x2"`, then the message included with the exception would be:
`invalid literal for int() with base 10: '35x2'`

EXTRA: Output Messages (2)

- To obtain the message, we must have access to the exception object itself
- You can store the exception object in a variable with the **as** syntax:

```
except ValueError as exception :
```

- When the handler for ValueError is executed, **exception** is set to the exception object. In our code, we then obtain the message string by calling **str(exception)**

Syntax: Try-Except

Syntax

```
try :  
    statement  
    statement  
    . . .  
except ExceptionType :  
    statement  
    statement  
    . . .  
except ExceptionType as varName :  
    statement  
    statement  
    . . .
```

This function can raise an
IOError exception.

```
try :  
    infile = open("input.txt", "r")  
  
    line = inFile.readline()  
    process(line)
```

When an IOError is raised,
execution resumes here.

```
except IOError :  
    print("Could not open input file.")
```

Additional except clauses
can appear here. Place
more specific exceptions
before more general ones.

```
except Exception as exceptObj :  
    print("Error:", str(exceptObj))
```

This is the exception object
that was raised.

The finally Clause

- The **finally** clause is used when you need to take some action whether or not an exception is raised
- Here is a typical situation
 - It is important to **always close an output file** whether or not an exception was raised (to ensure that all output is written to the file)
 - Place the call to **close()** inside a **finally** clause:

```
outfile = open(filename, "w")
try :
    writeData(outfile)
finally :
    outfile.close()
```

Syntax: The Finally Clause

Advanced

Syntax

```
try :  
    statement  
    statement  
    . . .  
finally :  
    statement  
    statement  
    . . .
```

```
outfile = open(filename, "w")  
try :  
    writeData(outfile)  
    . . .  
finally :  
    outfile.close()  
    . . .
```

This code may raise exceptions.

This code is always executed, even if an exception is raised in the try block.

The file must be opened outside the try block in case it fails. Otherwise, the finally clause would try to close an unopened file.

The With Statement

- Because a `try/finally` statement for opening and closing files is so common, Python has a special shortcut:

```
with open(filename, "w") as outfile :  
    Write output to outfile
```

- This `with` statement opens the file with the given name, sets `outfile` to the file object, *and closes the file object* when the end of the statement has been reached or an exception is raised

Detecting Errors

- What do you do if someone tries to withdraw too much money from a bank account?
- You can “**raise**” an exception
- When you **raise** an exception, **execution** does not continue with the next statement
 - It **transfers** to the **exception handler**

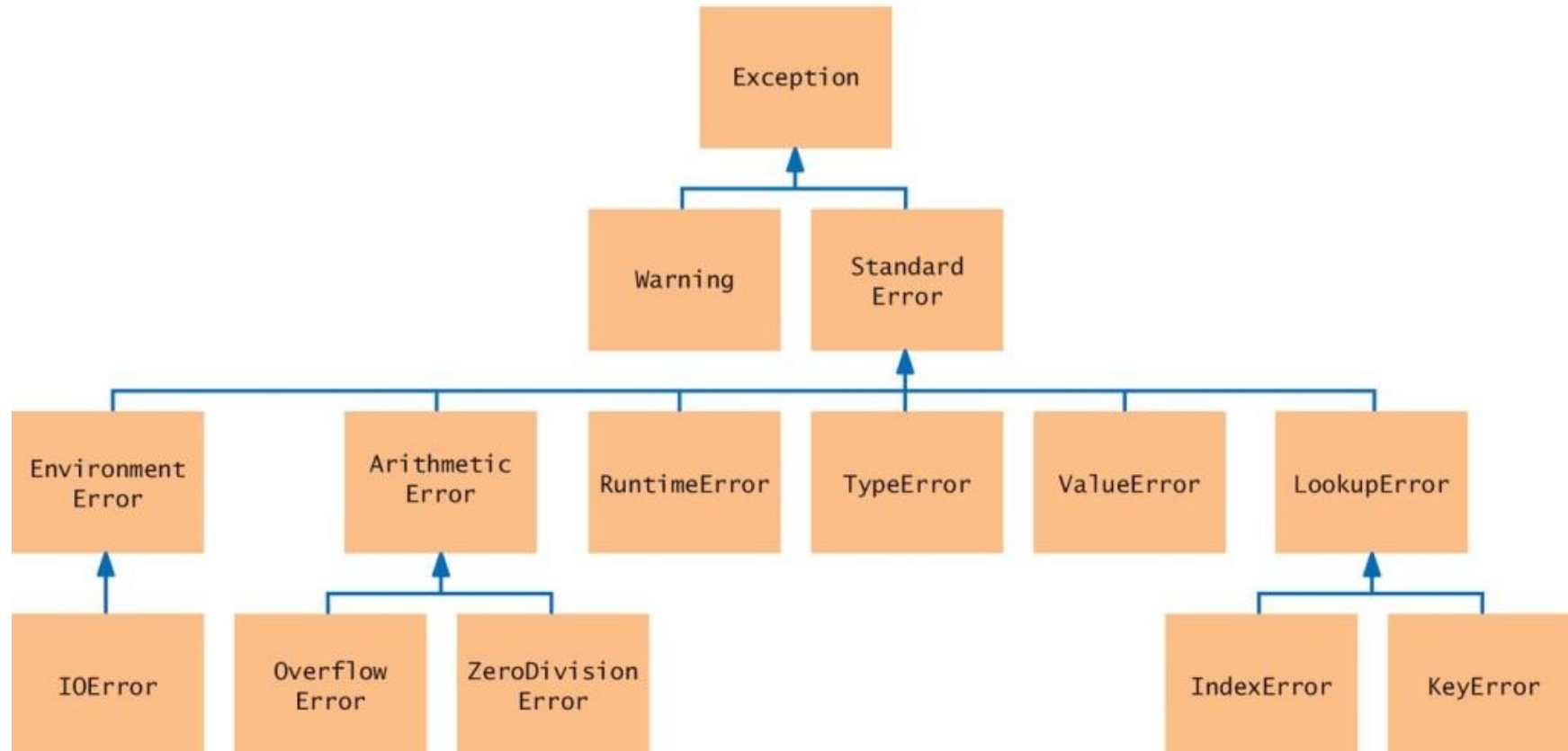
Use the **raise** statement to signal an exception

```
if amount > balance :  
    raise ValueError("Amount exceeds balance")
```

Exception Classes (a subset)

Advanced

- Look for an appropriate exception



Syntax: Raising an Exception

Advanced

Syntax `raise exceptionObject`

A new
exception object
is constructed,
then raised.

```
if amount > balance :  
    raise ValueError("Amount exceeds balance")  
  
balance = balance - amount
```

This message provides detailed
information about the exception.

This line is not executed when
the exception is raised.

Handling Input Errors



7.6

Handling Input Errors

- File Reading Application Example

- Goal: Read a file of data values

- First line is the count of values
- Remaining lines have values

```
3
1.45
-2.1
0.05
```

- Risks:

- The file may **not exist**

- The `open()` function will raise an exception when the file does not exist

- The file might have data in the wrong format

- When there are fewer data items than expected, or when the file doesn't start with the count of values, the program will raise a `ValueError` exception
- Finally, when there are more inputs than expected, a `RuntimeError` exception should be raised

Handling Input Errors: main()

- Outline for method with all exception handling

```
done = False
while not done :
    try:
        # Prompt user for file name
        data = readFile(filename) # May raise exceptions
        # Process data
        done = true;
    except IOError:
        print("File not found.")
    except ValueError :
        print("File contents invalid.")
    except RuntimeError as error:
        print("Error:", str(error))
```

Handling Input Errors: readFile()

- Creates the file object and calls the readData() function
- No exception handling (no except clauses)
- **finally** clause closes file in all cases (exception or not)

```
def readFile(filename) :  
    inFile = open(filename, "r") # May throw exceptions  
    try  
        return readData(inFile)  
    finally  
        in.close()
```

Handling Input Errors: readData()

- No exception handling (no try or except clauses)
- `raise` creates a `RuntimeError` exception and exits
- `ValueError` exception can occur

```
def readData(inFile) :  
    line = inFile.readline()  
    numberOfValues = int(line) # May raise a ValueError exception.  
    data = []  
    for i in range(numberOfValues) :  
        line = inFile.readline()  
        value = int(line) # May raise a ValueError exception.  
        data.append(value)  
    # Make sure there are no more values in the file.  
    line = inFile.readline()  
    # Extra data in file  
    if line != "" :  
        raise RuntimeError("End of file expected.")  
    return data
```

One Possible Scenario

- main calls readFile
 - readFile calls readData
 - readData calls int
 - There is no integer in the input, and int raises a ValueError exception
 - readData has no except clause; it terminates immediately
 - readFile has no except clause; it terminates immediately after executing the finally clause and closing the file
- The IOError except clause is skipped
- The ValueError except clause is executed

Summary

Summary: File Input/Output

- When opening a file, you supply the name of the file stored on disk and the mode in which the file is to be opened
- Close all files when you are done processing them
- Use the `readline()` method to obtain lines of text from a file
- Write to a file using the `write()` method or the `print()` function

Summary: Processing Text Files

- You can iterate over a file object to read the lines of text in the file
- Use the `rstrip()` method to remove the newline character from a line of text
- Use the `split()` method to split a string into individual words
- Read one or more characters with the `read()` method

Summary: Exceptions (1)

- To signal an exceptional condition, use the **raise** statement to raise an exception object
- When you **raise** an exception, processing continues in an exception handler
- Place the statements that can cause an exception inside a **try** block, and the handler inside an **except** clause
- Once a **try** block is entered, the statements in a **finally** clause are guaranteed to be executed, whether or not an exception is raised

Summary: Exceptions (2)

- Raise an exception as soon as a problem is detected
 - Handle it only when the problem can be handled
- When designing a program, ask yourself what kinds of exceptions can occur
- For each exception, you need to decide which part of your program can competently handle it

Thanks

- Part of these slides are [edited versions] of those originally made by **Prof Giovanni Squillero** (Teacher of Course 1)
- Part of these slides are [edited versions] of those originally made by **Prof Massimo Poncino** (Teacher of one of the courses in Italian)

License

- These slides are distributed under the Creative Commons license “Attribution - Noncommercial - ShareAlike 2.5 (CC BY-NC-SA 2.5)”
- You are free:
 - to reproduce, distribute, communicate to the public, exhibit in public, represent, perform and recite this work
 - to modify this work
- Under the following conditions:
 - **Attribution** — You must attribute the work to the original authors, and you must do so in a way that does not suggest that they endorse you or your use of the work.
 - **Non-Commercial** — You may not use this work for commercial purposes.
 - **Share Alike** — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same or equivalent license as this one.
- <http://creativecommons.org/licenses/by-nc-sa/2.5/it/>

